

CMP(N)403 Languages and Compilers - Phase II Report

Project Overview

This project implements a compiler for a subset of the C++ programming language. The compiler evaluates source code to enforce proper lexical, syntactic, and semantic rules, ultimately generating a list of intermediate representations known as quadruples.

In **Phase II**, the compiler's capabilities have been extended beyond basic lexical and syntactic analysis to include:

- **Symbol Table Management:** An extensible symbol table that handles nested block scopes and function parameters, tracking variable types, mutability, initialization status, and usage.
- **Semantic Analysis:** Strict semantic enforcement including checking variable declarations (to prevent **variable redeclarations** or **usage of undeclared variables**), validating type compatibility in assignments and returns, enforcing constant immutability, and ensuring proper **control-flow context** (e.g., `break` and `continue` only inside loops).
- **Error Recovery:** A robust syntax error handler utilizing Yacc's `error` token to allow the compiler to recover from **localized parse errors** and continue analyzing the file, yielding a comprehensive list of all errors present rather than **halting at the first syntax mistake**.
- **Intermediate Code Generation:** The systematic generation of Quadruples for evaluated expressions, assignments, conditional logic (if/else/switch), loops (while/do-while/for), and function calls.

Tools and Technologies Used

- **Lexical Analyzer Generator:** `Flex` is used to modularize the grammar and generate the scanner (`lexer.l`).
- **Parser Generator:** `Bison` is used to generate the parser (`parser.y`) that forms the syntactic and semantic of the compiler.
- **Implementation Language:** `C` language. Data structures and algorithms for quadruples and the symbol table are implemented natively in `C`.
- **Build System:** `GNU Make` simplifies compilation, linking, and executing unit tests for the compiler suite.
- **Testing environment:** Custom `.gpp` files demonstrating all working functionality in a UNIX-like environment.

Tokens List

The lexer (`lexer.l`) generates the following tokens to feed into the parser:

Data Types and Keywords

Token	Description
INT_TYPE	int data type keyword
FLOAT_TYPE	float data type keyword
DOUBLE_TYPE	double data type keyword
CHAR_TYPE	char data type keyword
BOOL_TYPE	bool data type keyword
VOID_TYPE	void data type keyword
CONST_KW	const keyword for constant variable definitions
IF, ELSE	Conditional branching keywords
WHILE, DO, FOR	Loop control keywords
SWITCH, CASE, DEFAULT	Switch statement branching keywords
BREAK, CONTINUE	Loop and switch flow control keywords
RETURN	Function return keyword
TRUE_KW, FALSE_KW	Boolean literal keywords

Literals and Identifiers

Token	Description
INTEGER_LITERAL	Represents an integer number (e.g., 5, 42)
FLOAT_LITERAL	Represents a floating-point number (e.g., 3.14)
CHAR_LITERAL	Character literal enclosed in single quotes (e.g., 'a')
STRING_LITERAL	String literal enclosed in double quotes (e.g., "Hello")
IDENTIFIER	Represents a valid variable or function name

Operators

Token	Description
PLUS, MINUS, MULT, DIV, MOD	Arithmetic operators (+, -, *, /, %)
INC, DEC	Increment (++) and decrement (- -) operators
EQ_OP, NE_OP	Equality (==) and inequality (!=)
LT_OP, GT_OP	Relational less than (<) and greater than (>)
LE_OP, GE_OP	Relational less than or eq (<=) and greater than or eq (>=)
AND_OP, OR_OP, NOT_OP	Logical operators (&&, `
ASSIGN	Direct assignment operator (=)
ADD_ASSIGN, SUB_ASSIGN	Compound assignment operators (+=, -=)
MUL_ASSIGN, DIV_ASSIGN, MOD_ASSIGN	Compound assignment operators (*=, /=, %=)

Syntax and Punctuation

Token	Description
LPAREN, RPAREN	Grouping parentheses (and)
LBRACE, RBRACE	Block scopes and compound statements { and }
LBRACKET, RBRACKET	Array indexing [and]
SEMI	End of statement delimiter ;
COMMA	Variable separation or argument delimiter ,
COLON	Used for case and default statements :

Quadruples List

The compiler uses a three-address code structure formatted as `(Operator, Arg1, Arg2, Result)`. The following table details the generated quadruples covering the required C++ constructs.

Quadruple / Operator	Description	Structure
ASSIGN	Assigns the value of Arg1 to the Result.	(ASSIGN, Value, -, Destination)
ADD, SUB	Adds/Subtracts Arg1 and Arg2, storing in Result.	(ADD, Arg1, Arg2, Result) (SUB, Arg1, Arg2, Result)
MUL, DIV, MOD	Multiplies/Divides/Modulo of Arg1 and Arg2 into Result.	(MUL, Arg1, Arg2, Result) (DIV, Arg1, Arg2, Result) (MOD, Arg1, Arg2, Result)
NOT	Logical negation of Arg1, storing in Result.	(NOT, Arg1, -, Result)
EQ, NE, LT, GT, LE, GE	Tests relation between Arg1 and Arg2.	(LT, Arg1, Arg2, Result) (EQ, Arg1, Arg2, Result)
LABEL	Defines a jump destination or control flow target.	(LABEL, -, -, LabelName)
JMP	Unconditional jump to the Result label.	(JMP, -, -, LabelName)
JMP_TRUE	Jumps to Result label if Arg1 evaluates to true.	(JMP_TRUE, Condition, -, LabelName)
JMP_FALSE	Jumps to Result label if Arg1 evaluates to false.	(JMP_FALSE, Condition, -, LabelName)
PARAM	Pushes Arg1 as an argument prior to a function call.	(PARAM, ParamValue, -, -)
CALL	Calls the function Arg1 with Arg2 arguments. Returns to Result if not void.	(CALL, FunctionName, ArgCount, Result)

RETURN	Returns Arg1 from the current function back to the caller.	(RETURN, ReturnValue, -, FunctionName)
FUNC_BEGIN	Marks the entry point for a function body logic.	(FUNC_BEGIN, FunctionName, -, -)
FUNC_END	Marks the termination of a function body logic.	(FUNC_END, FunctionName, -, -)